

53

Defn A Known-plaintext attack is an attack where an adversary has access to some plaintexts and the corresponding ciphertexts.

Ex: Assignment 1, Q4.

Suppose that a bit stream is encrypted using an LFSR of length L .

Further suppose that an attacker knows $2L$ consecutive bits of plaintext $x_0, \dots, x_{2L-1}$ and also the corresponding bits of ciphertext $y_0, \dots, y_{2L-1}$.

The attacker then obtains $2L$ consecutive bits

$$s_0, \dots, s_{2L-1}$$

from the key stream, where $s_j = x_j + y_j$ (in $\mathbb{Z}_2$).

Suppose that the attacker wishes to find the connection polynomial

$$c(x) = 1 + c_1 x + \dots + c_L x^L$$

of the LFSR.

54

We obtain the following system of linear equations in the L variables $c_1, \dots, c_L$ over $\mathbb{Z}_2$:

$$\begin{aligned}
 s_L &= c_1 s_{L-1} + \dots + c_L s_0 \\
 s_{L+1} &= c_1 s_L + \dots + c_L s_1 \\
 &\vdots \\
 s_{2L-1} &= c_1 s_{2L-2} + \dots + c_L s_{L-1}.
 \end{aligned}$$

In practice, an attacker can probably efficiently and uniquely solve this using Gaussian elimination over $\mathbb{Z}_2$. This will allow the attacker to recover the (connection polynomial of) the LFSR being used.

Matrix version: Our system above is equivalent to

$$\underbrace{\begin{bmatrix} s_{L-1} & s_{L-2} & \dots & s_1 & s_0 \\ s_L & s_{L-1} & \dots & s_2 & s_1 \\ \vdots & \vdots & & \vdots & \vdots \\ s_{2L-2} & s_{2L-3} & \dots & s_L & s_{L-1} \end{bmatrix}}_{=: A} \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_L \end{bmatrix} = \underbrace{\begin{bmatrix} s_L \\ s_{L+1} \\ \vdots \\ s_{2L-1} \end{bmatrix}}_{=: b}$$

Hence, if $\bar{A}$ denotes an inverse of A over $\mathbb{Z}_2$, then

55

$$\begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_L \end{bmatrix} = \bar{A}b.$$

Note: Use this for Assignment 2, Q3.

You can use the function modular-inverse-of-matrix to find $\bar{A}$ and then use modular-matrix-product to compute $\bar{A}b$.

Both functions are defined in the cs40d Python module.

Combining LFSRs: In practice, several LFSRs are usually combined using (non-linear) combining functions.

Ex: "A5/1" (GSM/2G mobile phone) is based on 3 LFSRs of lengths 19, 22, and 23, respectively. The three key streams are added (in $\mathbb{Z}_2$). However, "clocking bits" are used to decide which registers are clocked at each step.

Details: $\rightarrow$ Smart, pp 118-119

Attacks: • A first attack was proposed in 1994.

• Completely broken according to the 2013 NSA leaks.

Public-Key cryptography

Recall: a (symmetric) cryptosystem consists of

- sets P, C, K of plaintexts, ciphertexts, and keys

and

- for each key $k \in K$, functions

$$e_k: P \rightarrow C \quad (\text{"encryption"})$$

$$d_k: C \rightarrow P \quad (\text{"decryption"})$$

with

$$d_k(e_k(x)) = x$$

for all $x \in P$.

In practice: Alice and Bob somehow (e.g. by meeting in person) agree on a common key which they then use to encrypt/decrypt messages between them.

Issues:

- How can Keys be exchanged securely?
- Key distribution scales poorly:

Suppose that in a group of $n \geq 2$ people, any two wish to be able to communicate securely with each other.

Then:

$$\binom{n}{2} = \frac{n(n-1)}{2}$$

Keys need to be exchanged.

Ex: To allow any two of the $\approx 18,000$ students of NUI Galway to communicate securely,

$$\approx \binom{18,000}{2} = 161,991,000$$

Keys would need to be generated and exchanged.

This is completely infeasible!

58
(Remark: we could use a centralized Key exchange server but then we would need to trust whoever is running that server!)

Enter public-key cryptography!

Idea:

(published by W. Diffie, M. Hellman 1976;
previously contemplated by J.H. Ellis (@GCHQ)
in 1970)

Use different keys for encryption and decryption.

In greater detail:

- Bob generates a matching pair of keys, one for encryption and one for decryption.
- The encryption key is made public ("public key"), the decryption key is kept secret ("private key").
- If Alice wishes to send a message to Bob, she obtains a copy of his public key and uses it to encrypt her message.
- Bob can then decrypt the message using his private key.

Crucial: it should be infeasible to recover Bob's private key from his public one.

(It should also be infeasible to decrypt messages without the private key.)

Defⁿ A function is a one-way function if it can be efficiently computed for all inputs but if it is computationally hard to invert the function (i.e. to construct preimages).

Hope: find a one-way function which can be efficiently inverted iff the private key is known and use it for encryption.

Open (!) problems:

- ① Do one-way functions even exist?
- ② Is there a provably secure public-key cryptosystem?

(Recall: one-time pads provide provably secure symmetric cryptosystems.)

Practical public-key cryptography

61

Dense public-key cryptosystems based on mathematical problems involving functions which are easily computable but which seem to be hard to invert (i.e. we don't know how to do it efficiently and we suspect it can't be done).

Main sources of mathematical problems:

- Integer factorisation (e.g. RSA)
- Discrete logarithms (e.g. ElGamal, Diffie-Hellman key exchange)
- Elliptic curves

Words of wisdom?

"No one has yet discovered any warlike purpose to be served by the theory of numbers or relativity, and it seems unlikely that anyone will do so for many years."

— G. H. Hardy, "A Mathematician's Apology" (1941)

Remarks:

- Since public Keys are... public, Key distribution is not a problem (put them on your web page, Facebook profile, etc.).

However, Key authenticity can be an issue:

how does Alice know that she really received Bob's public Key and not somebody else's?

- In practice, symmetric and public-key cryptosystems are often combined:
 - Alice generates a random Key ("session Key") for a symmetric crypto system and encrypts her message.

- She then encrypts the session key using Bob's public key and sends both the encrypted session key and the previously encrypted message.
- Bob first decrypts the session key using his private key and then decrypts the actual message using the session key.

Reason for combining symmetric & public-key cryptography:

symmetric cryptosystems are usually much more efficient

let's return to the formal definition of symmetric cryptosystems.

Defⁿ A public-key cryptosystem consists of sets P, C, K as before and functions $enc_k: P \rightarrow C$ and $dec_k: C \rightarrow P$ for $k \in K$ with the following property:
 for each (public key) $e \in K$, there exists (a private key) $d \in K$ with

$$\boxed{dec_d(enc_e(x)) = x} \text{ for all } x \in P.$$

The RSA cryptosystem

65

- Developed by R. Rivest, A. Shamir, and L. Adleman 1977-1978.

- An equivalent system was developed by C. Cocks (@ GCHQ). This was classified.

His work became publicly known in 1997.

At the time, GCHQ saw no practical use for the cryptosystem.

- RSA was the first and remains the most important public-key cryptosystem.

We'll first describe the different parts of RSA and then study how and why it works.

Description of RSA

• Key generation:

Alice picks two (different, large, random) primes p and q and computes their product

$$n = p \cdot q.$$

She then chooses a (random) integer e with $1 < e < (p-1)(q-1)$ and $\gcd(e, (p-1)(q-1)) = 1$.

Her public key is (n, e) .

Next, using the extended Euclidean algorithm, she computes the unique integer d with $1 < d < (p-1)(q-1)$ and $de \equiv 1 \pmod{(p-1)(q-1)}$.

(In other words, $d \in \mathbb{Z}_{(p-1)(q-1)}^{\times}$ is the inverse of e modulo $(p-1)(q-1)$.)

Alice's private key is (n, d) .

Note: the primes p, q are not part of either key.
They need to be kept secret since d can be effectively computed from p, q, e .

- Plaintexts and ciphertexts are the elements of $\mathbb{Z}_n$.
- Encryption:

Given a message $m \in \mathbb{Z}_n$, Bob uses Alice's public key (n, e) to compute the ciphertext

$$c = m^e \bmod n.$$

- Decryption:

After receiving c , Alice uses her private key (n, d) to decrypt

$$m = c^d \bmod n.$$